SAEM

EdTech Research & Codespaces Build Book

A research-backed blueprint for a scientific learning platform, adaptive cognition, verified problem solving, and original visual explanations

JEE Advanced • NEET • Olympiads • University Physics • Mathematics

VISION → RESEARCH → ARCHITECTURE → TESTABLE MVP

Research edition • 22 October 2026 • Designed for implementation in GitHub Codespaces

How to use this book

This is an implementation-oriented research specification, not a promise that an AI system can autonomously teach or verify every scientific idea. It converts the SaemPhysics production-studio vision into a broader Saem EdTech platform with three deliberately separate engines: scientific correctness, learner modeling, and visual explanation.

Read Chapters 1–6 to settle product and research decisions. Chapters 7–12 define the data, verification, visual compiler, and engineering boundaries. Chapters 13–16 turn the design into a Codespaces work plan and measurable release gates. The appendices provide a repository skeleton, schemas, prompts, and a book-ingestion protocol.

SOURCE BOUNDARY AND HONESTY NOTE

The workspace available for this task contains two earlier SaemPhysics planning documents: “AI Animation Studio: Architecture, Engineering & Production Workflow Textbook” and “Future Roadmap: Knowledge-to-Lecture Engine.” These are used as the project baseline. The separate reference books mentioned as uploaded to your profile were not visible as files in this conversation, so this edition does not pretend to quote or analyze them. Chapter 15 specifies exactly how to integrate those books when the files are attached. The literature list is based on established works known to the authoring model; links and editions should be verified before formal publication.

Research questions this book is designed to answer

- **Product:** How can a question library, concept graph, adaptive learner model, scientific solver, and animation compiler become one coherent teaching platform?
- **“Quantum Cognitive Algorithm”:** What could that phrase mean precisely, and how should a non-classical or quantum-inspired model be tested rather than assumed to be better?
- **Trust:** How can the platform prevent plausible-looking but incorrect physics, invented citations, rights violations, and unreviewed generated lessons?
- **Engineering:** How can the frontend be authored mainly in Python while the runtime backend remains C#, C++, and/or C?
- **Execution:** What is the smallest end-to-end feature that proves the architecture before the project grows too large?

Contents

PART I — Product thesis: 1. North star; 2. Three-engine architecture; 3. Language and deployment strategy

PART II — Learning intelligence: 4. Measurable learner model; 5. Define and test the Quantum Cognitive Algorithm

PART III — Scientific knowledge to lessons: 6. Exam-question ingestion; 7. Scientific verification; 8. Pedagogy; 9. Visual lesson compiler

PART IV — Build specification: 10. Data model and service contracts; 11. Codespaces workflow; 12. JEE mechanics vertical slice

13. Evaluation and release gates; 14. Twelve-month roadmap and risks; 15. Reference-book synthesis; 16. Governance and privacy

APPENDICES — A. Repository tree; B. Domain records; C. Coding-agent prompt; D. Acceptance tests; E. Bibliography; F. Glossary

PART I — Product thesis and system boundaries

1. The Saem EdTech north star

1.1 Product thesis

Saem EdTech should not begin as a giant “AI tutor that knows everything.” It should begin as a trusted scientific learning workbench: every question has a traceable source record; every computational answer has explicit assumptions and checks; every learner recommendation has a measurable basis; and every visual explanation is generated from a reviewable lesson specification.

The product promise is not “AI replaces a teacher.” The promise is that a teacher can create, verify, adapt, and reuse high-quality explanations faster, while students receive practice and feedback that respond to evidence of their understanding.

1.2 Target users and jobs to be done

User	Primary need	Platform response	What success looks like
Student	Understand a concept, solve a problem, know what to study next	Stepwise solution, visual model, diagnostic question, next-practice recommendation	Better transfer to new questions; durable recall; fewer repeated misconceptions
Teacher / creator	Create accurate lessons and reusable explanations	Source panel, solver checks, scene editor, review and release workflow	Less production friction without loss of scientific control
Content editor	Maintain a large question bank with provenance	Metadata, rights state, duplicate detection, review queue	Each item traceable, permissioned, and auditable
Researcher / platform owner	Evaluate whether personalisation improves outcomes	Experiments, baseline comparisons, versioned metrics	Measured learning gain rather than unsupported marketing claims

1.3 Design invariants

- **Truth must not depend on the learner model.** The learner model can choose which explanation to show; it cannot alter the correct physics answer.
- **Visual appeal must not overrule correctness.** A rendering can be beautiful and still be rejected if its model, labels, vectors, units, or timing are wrong.
- **Generative models draft; deterministic tools check.** A language model may propose a derivation, but the system should independently test symbolic relations, numerical cases, dimensions, and edge cases.
- **Every source-derived claim needs provenance.** Record source, location, transformation, licence or permission status, and human reviewer.
- **Uncertainty must remain visible.** When extraction, answer keys, model fit, or physics interpretation is ambiguous, the system must mark the item for review rather than silently decide.
- **Automate narrow tasks first.** The first milestone is one complete, reviewable question-to-lesson path—not autonomous ingestion of thirty years of all subjects.

1.4 Product scope: what to build first and what to defer

Build first	Defer until evidence exists
One topic: JEE Advanced mechanics, initially 20–50 curated items	Bulk ingestion of all exams and subjects
Deterministic solvers for a small set of explicit problem families	A universal physics solver that claims to understand every problem
A concept graph reviewed by the teacher	A fully automated ontology inferred from raw questions

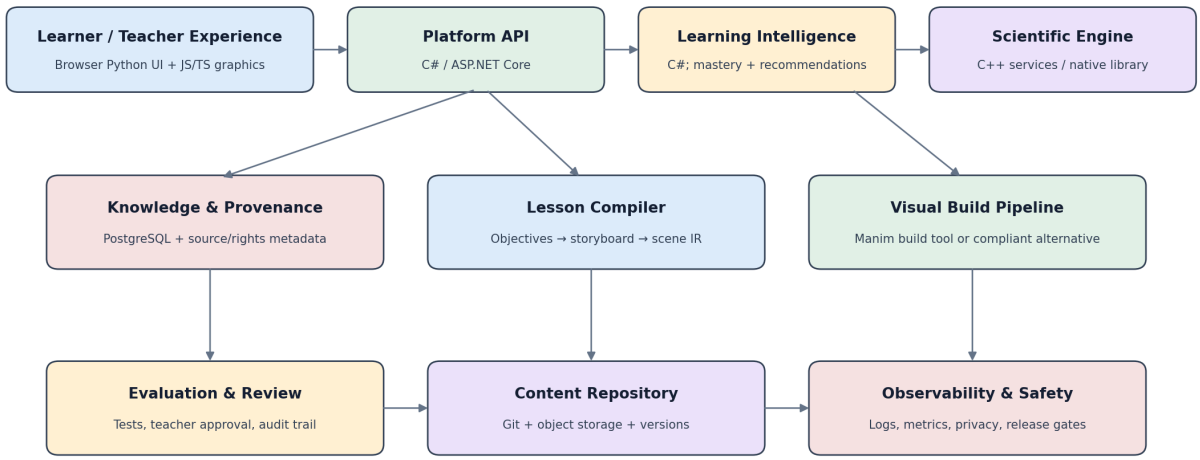
Build first	Defer until evidence exists
Classical learning model with transparent baselines	Quantum-labelled production recommendation without evaluation
One structured storyboard and one renderer	One-click auto-generated long-form videos
Manual approval and traceable revisions	Fully autonomous publishing

STRATEGIC RECOMMENDATION

Organise the product around a narrow, high-quality vertical slice. Scale the content volume only after the architecture proves provenance, correctness, repeatability, and learner value.

2. The three-engine architecture

Keep three engines distinct and let the platform coordinate them. This boundary is more important than any individual language or framework.



Invariant: learning recommendations, scientific truth, and visual storytelling are separate concerns.

2.1 Scientific Intelligence Engine

Purpose: determine what follows from the declared model and inputs. It owns symbolic algebra, numerical simulation, dimensional analysis, equation solving, uncertainty propagation where relevant, and verification reports. It should return a result plus assumptions, method identifiers, warnings, and machine-readable checks—not only a final number.

- Inputs are typed problem records, not unstructured chat alone.
- Outputs include values with units, intermediate expressions, assumptions, solver version, and test evidence.
- Ambiguous modeling choices trigger a clarification or a flagged review state.
- Independent checks should use a genuinely different route when practical: analytic derivation versus numeric evaluation, conservation law versus time integration, or one solver implementation versus a second formulation.

2.2 Learning Intelligence Engine

Purpose: estimate a learner's likely mastery of concepts and choose the next instructional action. It may use question outcomes, hint requests, time-on-task with caution, self-explanations, repeated errors, and delayed recall. It does not infer a child's identity or mental state beyond what is needed for learning, and it should communicate uncertainty.

- Keep concept mastery separate from question difficulty and test performance.
- Store evidence events rather than overwriting a single opaque "ability score."
- Treat response correctness as noisy evidence: guessing, careless slips, ambiguous items, and external help are possible.
- Offer a teacher override and a "why this recommendation?" explanation.

2.3 Visual Intelligence Engine

Purpose: explain one idea through time, geometry, graphs, symbols, and narration. It converts a lesson specification into an animation, interactive diagram, or narrated video. It never decides scientific truth independently; it consumes approved equations, ranges, units, story beats, and assertions from the scientific engine.

- The intermediate scene format must be renderer-independent.
- Every scene has one learning purpose and one key visual claim.
- Motion should expose structure, not decorate the screen.
- A scene regression test checks coordinates, parameter values, labels, and visual invariants where possible.

2.4 Platform shell and governance services

The platform shell handles accounts, roles, courses, assignments, repositories, permissions, review queues, notifications, job status, audit logs, and release workflow. It is not another reasoning engine. The platform makes the engines usable and accountable.

Concern	Owner	Boundary
Authentication, roles, course enrolment	C# platform API	Never store passwords or credentials in the repository
Question sources and rights	Knowledge/provenance service	An item with unknown rights can be used privately for review but not assumed publishable
Scientific calculations	C++ scientific engine; C# adapter	Return typed results and structured failures
Learner state and recommendations	C# learning service	Version each model and preserve evidence history
Scene building	Compiler + renderer adapter	Consume a validated scene IR; reject malformed specs
Publishing	Release orchestrator	Requires checks and explicit human approval for consequential outputs

3. Language and deployment strategy

3.1 The requirement and the trade-off

Your stated preference is a Python-authored frontend, a C/C++/C# backend, and modern graphics where Python alone would be limiting. This is possible in a hybrid architecture, but "frontend written in Python" needs an exact definition: Python can execute in a browser runtime, or it can generate server-side HTML through a Python server. The second choice would violate a strict no-Python-backend rule.

Layer	Recommended starting choice	Why	Constraint
Browser application logic	Python running client-side via Pyodide/PyScript-style runtime	Keeps much UI logic authored in Python	Browser startup, package compatibility, and debugging may be harder than TypeScript
High-performance graphics	TypeScript/JavaScript + Canvas/WebGL/WebGPU as appropriate	Modern browser graphics ecosystem and responsive interaction	Use a stable interface so graphics implementation can evolve separately
Main API and platform services	C# / ASP.NET Core	Strong fit for auth, typed domain services, APIs, background orchestration	Requires clean API contracts and automated integration tests
Numerical/scientific hot paths	C++ library or service, exposed through stable C ABI or service protocol	Suitable for performance-sensitive algorithms and native libraries	Avoid passing complex object graphs across a native boundary
Persistent storage	PostgreSQL + versioned object storage	Transactions and structured metadata plus large media/source files	Backups, migrations, permissions, and retention need explicit policy
Animation build	Isolated build worker using Manim, if Python is allowed as tooling	Leverages established mathematical animation workflows	If no Python process is allowed anywhere server-side, use a non-Python renderer or build videos outside the runtime platform

3.2 Recommended architecture under a strict runtime-backend rule

1. Browser: Python-authored UI modules run client-side; JavaScript/TypeScript components handle demanding graphics and browser integration.
2. Runtime API: C# only for authentication, courses, question services, learner events, recommendations, jobs, and release workflows.
3. Scientific engine: C++ native library called through a small C ABI from C# for local deployments, or a versioned local/network service if isolation is preferred.
4. Batch authoring: animation/video generation runs as a controlled build tool. If policy defines every rendering worker as a backend, Manim is not compliant with the strict language rule; explicitly approve the exception or choose a different renderer.
5. Communication: OpenAPI/JSON for service contracts; Protobuf/gRPC only if profiling shows it is useful. Start with the simpler boundary.

3.3 Why not make every component C++?

The goal is reliable product delivery, not maximum language uniformity. C# is a reasonable home for product workflows and application security; C++ is justified for measured numerical or performance needs; browser graphics can use the web platform. Prematurely moving all business logic to C++ increases development and safety burden without automatically improving learning outcomes.

3.4 Codespaces as the development environment

Use GitHub Codespaces as a reproducible development shell, not as a substitute for architecture. A devcontainer should install pinned development tools, expose only needed ports, run formatting and tests, and document any GPU or video-rendering limitation. Large renders can later move to a dedicated worker or CI runner; source code and small test renders should remain reproducible from the repository.

- Pin dependency versions and capture the runtime versions in build artifacts.
- Never commit API keys, model-provider secrets, exam scans with unclear rights, student records, or large generated media by default.
- Use small deterministic fixtures in continuous integration; reserve long video renders for an explicit build job.
- Make one command run the unit test suite and another command run the end-to-end demonstration.

PART II — Learning intelligence and the quantum-cognition research track

4. A measurable learner model

4.1 Start with a transparent baseline

The initial cognitive engine should be a transparent, falsifiable model, not a branded “quantum” formula. Start with a reviewed concept graph and a simple learner-state model. Candidate baselines include Bayesian Knowledge Tracing (BKT) for skill mastery, Item Response Theory (IRT) for item/learner response patterns, and an explicitly evaluated forgetting/review scheduler. These models answer different questions and should not be collapsed into one unexplained score [R4, R5].

Signal or model	What it contributes	What it cannot establish alone
Concept graph	Prerequisite relationships; links questions to skills	Whether a particular student has mastered a node
BKT	Posterior estimate of mastery from a sequence of responses	A complete psychological description or guaranteed individual diagnosis
IRT	Separates item difficulty and discrimination from learner response tendencies	Conceptual understanding of every skill; quality depends on model assumptions and data
Forgetting / spaced review	Schedules retrieval practice based on previous evidence and elapsed time	Exact biological memory state or a universal optimal interval
Error and hint features	Can identify likely misconception patterns and support targeted feedback	Causal proof that the learner holds a misconception

4.2 Concept graph design

Represent each concept as a versioned node with a definition, learning objectives, prerequisite edges, equations, representations, canonical question families, misconceptions, source references, and teacher review status. A directed edge such as “requires” should include a strength or confidence and a justification. Prerequisites are hypotheses to validate against curriculum expertise and learner data, not immutable truths.

Example chain for mechanics: vectors → position and displacement → velocity → acceleration → Newton’s laws → work-energy methods → momentum and collisions → rotation. Cross-links matter: energy methods and Newton’s laws are alternative solution tools for many problems rather than a simple one-way ladder.

4.3 Bayesian Knowledge Tracing as a starting model

Classic BKT represents mastery as a hidden state with a probability of knowing a skill. Its commonly described parameters are initial mastery, transition/learning probability, slip probability, and guess probability [R4]. In production, use the model as an estimate whose calibration must be tested, not as a truth label.

CONCEPTUAL UPDATE RULE

Given a prior mastery probability P(K), a response likelihood under “known” and “not known,” and an explicit transition model, update the posterior P(K | response). Keep the parameters versioned and fit them only when enough appropriate data exists. Do not copy a formula into production without tests for edge cases, missing responses, item ambiguity, and calibration.

4.4 Question tags need confidence and evidence

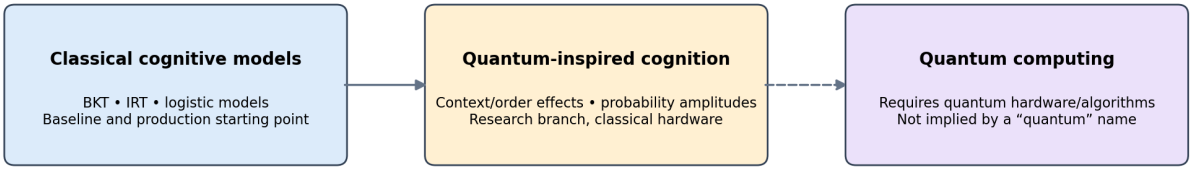
A question can involve several concepts, but the platform should distinguish primary skill, supporting skill, prerequisite, and incidental context. Store mapping confidence and the source of the decision (teacher, rule, model suggestion). A model-generated tag is a draft; it should not silently become ground truth.

4.5 Personalisation without manipulation

- Show the student why a concept or practice set is recommended.
- Allow “too easy,” “too hard,” and “this explanation did not help” feedback.
- Use time-on-question as a weak signal because reading speed, interruptions, accessibility needs, and device latency vary.
- Do not infer sensitive traits or use dark patterns to maximise time spent. Optimize for learning outcomes, not endless engagement.
- Track calibration: when the model predicts 70% success, roughly that rate should occur in comparable evaluated cases.

5. Define and test the Quantum Cognitive Algorithm

5.1 Resolve the phrase before coding it



Promotion rule: outperform transparent baselines on held-out learners and questions, with calibration and reproducibility.

“Quantum cognition” often refers to using the mathematics of quantum probability to model human judgments, decisions, contextuality, or order effects. It does not necessarily use a quantum computer. “Quantum-inspired” algorithms can run on classical hardware. Quantum computing is a separate claim about algorithms and hardware. These three meanings must not be conflated [R6, R7].

5.2 A research hypothesis, not a product promise

Working hypothesis Q1: in some learning contexts, response probabilities may depend on question order or contextual framing in a way that a particular classical model captures poorly, and a quantum-probability model may predict held-out behavior better. This is a research hypothesis to test; it is not established as a general advantage for adaptive education. The platform should not advertise an accuracy or learning advantage before evidence supports it.

5.3 A fair comparison protocol

6. Choose a narrowly defined outcome first: for example, predict whether a learner will correctly answer a new question from a specific concept family, or whether a delayed-retention check will be passed.
7. Create a data dictionary and freeze the train/validation/test split by learner and time. Prevent near-duplicate questions and future responses from leaking into training.
8. Compare against strong and transparent baselines: prevalence-only, logistic regression, BKT, and/or IRT as appropriate to the outcome.

9. Implement the quantum-inspired candidate with its assumptions documented mathematically. State what it can represent that the baseline cannot.
10. Use the same held-out data, comparable tuning effort, and documented compute budget. Predefine primary metrics and stopping criteria.
11. Report predictive quality, calibration, subgroup performance, robustness to sparse histories, latency, maintainability, and failure modes—not accuracy alone.
12. Repeat across topics and cohorts before making claims. A tiny improvement on one dataset may be noise or overfitting.

5.4 Metrics to report

Metric	Use	Caution
Log loss / cross-entropy	Quality of probabilistic predictions	Penalises overconfident wrong predictions; inspect alongside calibration
Brier score	Mean squared probabilistic error for binary outcomes	Can vary with event prevalence
Calibration plot / expected calibration error	Do predicted probabilities match observed outcomes?	ECE depends on binning; show plot and confidence intervals
AUC / PR-AUC	Ranking discrimination	Does not show calibration and may hide poor absolute predictions
Delayed-retention outcome	Learning durability after a planned interval	Needs a valid comparison design and adequate sample size
Teacher review burden	Operational cost of wrong or ambiguous recommendations	Define the unit of review and count disagreements consistently
Latency and complexity	Serving costs and maintainability	An academic model may be too expensive or opaque for production

5.5 Promotion gates for the quantum track

Gate	Evidence required	Decision
Q0: definition	Model specification, intended phenomenon, predicted advantage, failure conditions	Do not start coding until the claim is precise
Q1: reproducibility	Reference implementation, synthetic tests, fixed random seeds where applicable	Reject if results cannot be reproduced
Q2: offline benchmark	Held-out comparison against classical baselines; confidence intervals	Keep as research if no credible improvement
Q3: educational pilot	Ethically reviewed, consented pilot with a preregistered outcome when feasible	No causal claims without a sound design
Q4: production approval	Benefit exceeds complexity and privacy cost; monitoring and rollback exist	Deploy behind a feature flag, not as an irreversible architecture choice

RECOMMENDED DECISION

Build the product using classical models first, but design a pluggable model interface. The “quantum” research track should be allowed to win by evidence—or to remain a research experiment without delaying the core platform.

PART III — Scientific knowledge to original lessons

6. Thirty years of exam questions: ingest, map, and govern

6.1 Treat the exam archive as a rights-aware dataset

An exam question is both educational content and a data object with provenance and legal status. The fact that an old question appears online does not automatically make it public domain or free to republish. Store only what the platform is permitted to store and publish. Where rights are uncertain, retain source metadata and an internal review flag; do not distribute the scanned question or question text by default.

6.2 Ingestion pipeline

13. Discover and register the source: exam name, year, session, paper, source URL or repository location, retrieval date, licence/permission statement, and known restrictions.
14. Preserve the original privately and securely only where allowed. Calculate a checksum and record page or question location.
15. Extract text, maths, diagrams, and answer keys. Mark OCR confidence and preserve the source image for reviewer inspection where permitted.
16. Normalize into a typed question record without discarding the original transcription. Keep a diff between extracted text and reviewed text.
17. Detect duplicates and variants using source IDs, hashes, and semantic similarity; never auto-merge near-duplicates without review.
18. Attach candidate concepts and problem-family tags with confidence and evidence. Keep teacher-verified tags separate.
19. Run the appropriate solver family and verification suite. Preserve the official key, computed answer, and reviewer-approved answer as distinct fields.
20. Publish or expose only after rights, correctness, accessibility, and review gates pass.

6.3 Source and answer truth are separate dimensions

Field	Meaning	Rule
official_answer	What the official key says, with source location	Never overwrite; allow discrepancy annotations
computed_answer	What the implemented model/solver produced	Includes method, assumptions, units, checks, software version
reviewed_answer	Human-approved answer for the platform's stated interpretation	May differ from the official key if a documented key error exists
rights_status	Permission/licence state for storage, adaptation, and public display	Rights for ingestion, training, adaptation, and publication may differ
extraction_status	OCR/parser state and confidence	Low-confidence content blocks automatic solution generation
review_status	Draft, physics checked, rights checked, approved, retired	State transitions must be auditable

6.4 Question families are the bridge to a solver

Do not map every question straight to a single generic "AI solver." First classify the problem family: 1D constant acceleration, projectile motion, Newton-law free-body model, work-energy, momentum/impulse, circular motion, rigid-body rotation, gravitation, oscillation, etc. Each solver family should declare its supported assumptions, input schema, equations, and known failure modes.

6.5 Concept graph and Feynman map

For each question, attach: primary learning objective; required concepts; supporting concepts; common wrong turns; a “teach before solve” concept; the minimum mathematical prerequisites; and one possible visual metaphor. The metaphor is an explanatory hypothesis, not a source fact. It must be tested against student misunderstanding and scientific fidelity.

6.6 Licensing operations checklist

- Record rights for text, diagrams, official solutions, and derivative explanations separately where needed.
- Do not scrape at scale from sites when terms or robots/publisher restrictions forbid it.
- Do not assume that a question being an exam item makes it public-domain content.
- Keep source attribution attached to every adapted explanation.
- Use teacher-created variants and synthetic practice items to fill gaps when question-bank rights are uncertain.
- Keep a takedown/contact process and support retiring content without destroying the audit record.

7. Scientific verification: a multi-layer proof gate

7.1 Verification is layered evidence, not a single green tick

A unit test that checks a result against the same formula that generated it is a consistency check, not an independent proof. The Saem scientific engine should combine tests that fail for different reasons. Different checks should be represented separately and their scope stated clearly.

Layer	Example in mechanics	Failure caught	Limit
Input/model validation	Require units, valid mass/time, clear frame and assumptions	Malformed or underspecified problem	Cannot prove the selected physical model is appropriate
Symbolic check	Simplify the residual of an equation or verify derivative relations	Algebraic inconsistency	May reproduce assumptions or share a bug with source code
Dimensional check	Compare dimensions of every sum and equation	Unit and dimensional errors	Dimensionally valid expressions can still be physically wrong
Numerical test	Compare analytic solution with independent numerical integration over test cases	Implementation or sign errors	Numerics can hide errors at low precision or chosen cases
Invariant / limiting case	Zero acceleration, zero time, conserved energy/momentum where applicable	Boundary and model errors	Only checks the specific invariant/limit used
Alternate formulation	Energy method vs Newton-law solution when both apply	Shared implementation errors	Not independent if both routes share the same incorrect assumptions
Reference comparison	Compare against an approved worked solution or trusted benchmark	Mismatch with a known result	Reference itself may contain error or use different conventions
Human review	Teacher checks modeling, pedagogy, and interpretation	Contextual and conceptual errors	Needs a documented checklist and reviewer accountability

7.2 Verified result record

Every calculation should return a structure similar to this conceptual contract:

```
{
  "status": "verified_within_declared_checks",
  "problem_family": "constant_acceleration_1d",
  "assumptions": ["constant acceleration", "one-dimensional motion"],
  "result": {"symbol": "x", "value": 30.0, "unit": "m"},
  "checks": [
```

```
    {"id": "dimensions", "status": "passed"},
    {"id": "limiting_case_zero_time", "status": "passed"},
    {"id": "analytic_vs_numeric", "status": "passed"}
  ],
  "warnings": [],
  "solver_version": "mechanics-1d/0.1.0",
  "review_state": "awaiting_teacher_review"
}
```


7.3 Example: constant-acceleration motion

For the model $x(t) = x_0 + v_0t + \frac{1}{2}at^2$, a test set should include $t = 0$, $a = 0$, positive and negative initial velocity, units, and a domain with negative time only if the model permits it. To test more independently, compute velocity from the derivative and compare it with $v(t) = v_0 + at$; use a numerical integrator on selected parameters; and verify limiting cases. Avoid “verify” code that merely repeats the same expression with the same inputs and then compares the expression with itself.

7.4 Property-based and metamorphic testing

When exact expected values are scarce, test properties that should hold over many generated inputs. Examples: doubling all lengths and positions while holding time and velocity scales consistent should scale the output predictably; adding a constant offset to position should shift position by the same offset while leaving velocity unchanged; conservation laws should hold within a stated numeric tolerance in a closed model. Define tolerances and the domain of validity explicitly.

7.5 AI as a challenger, not the verifier

A language model can act as an adversarial reviewer by searching for hidden assumptions, sign errors, alternative solution methods, edge cases, or misleading wording. Agreement between two language models is not proof. If two models share training data or systematic biases, they can agree on the same wrong answer. The release gate must rely on typed models, executable checks, approved references, and human review.

8. Pedagogy: Feynman-inspired clarity without imitation

8.1 Convert intellectual traditions into explicit teaching methods

The system can encode teaching practices associated with conceptual explanation, mathematical structure, worked examples, active recall, and visual reasoning without copying a living creator’s distinctive presentation. The goal is a method library rather than a “write exactly like this person” feature.

Teaching principle	Executable design pattern	Evidence to collect
Explain the mechanism, not only the result	State the physical model, assumptions, and causal sequence before algebra	Learner can predict an outcome in a changed scenario
Move between representations	Connect verbal statement, equation, diagram, graph, and limiting case	Learner can translate between representations
Worked example followed by fading	Show a complete solution, then hide selected steps, then present a transfer problem	Independent success improves on a new item
Retrieval practice	Ask for a prediction or explanation before revealing the next step	Correct recall and retention after a delay
Spaced practice	Revisit concepts after increasing or evidence-based intervals	Delayed retention, not merely immediate fluency
Misconception contrast	Display a plausible wrong model next to a checked model and explain the difference	Reduced recurrence of the identified error
Alternate solution routes	Show when Newton’s laws, energy, or momentum is the efficient method and why	Learner chooses an appropriate method on new problems

Retrieval practice and distributed practice have a substantial evidence base in learning research [R1, R2, R3]. That does not mean any particular interface automatically produces those benefits. The platform must evaluate whether its implementation improves retention and transfer for its actual users.

8.2 A lesson specification

A high-quality lesson should contain a learning objective, target learner level, prior knowledge, central question, assumptions, conceptual model, misconception to confront, derivation, visual plan, worked example, transfer question, source record, and review state. A “Feynman-like” label should not replace these fields.

8.3 Audience adaptations

Mode	Prioritise	Avoid
NEET	Conceptual intuition, graphs, standard models, units, quick justified solution paths	Unexplained shortcuts or too many advanced branches
JEE Main	Model selection, robust algebra, standard traps, efficient reasoning	Memorising patterns without assumptions
JEE Advanced	Multiple constraints, hidden assumptions, alternate methods, limiting cases	Treating one method as universally best
Olympiad	Model construction, estimation, proof, unusual transfer, approximation validity	Reducing every task to a multiple-choice template
University / MIT-style	Derivation, structural mathematics, generalisation, links across topics	Using visual analogy as a replacement for proof

8.4 Don’t confuse fluency with understanding

An animation that makes a derivation feel obvious can still leave a learner unable to reproduce the reasoning. Place prediction questions before reveals, short retrieval tasks after an explanation, and a new scenario at the end. Record performance on those tasks rather than using views, completion, or satisfaction alone as proof of learning.

9. Visual lesson compiler and original animation grammar

9.1 Separate the lesson from the renderer

The lesson compiler should create a renderer-independent intermediate representation (IR). The IR contains semantic objects and relationships: equation, vector, graph, coordinate frame, parameter sweep, highlighted relation, narration cue, camera/panel layout, and accessibility text. A renderer adapter translates this IR into Manim or a browser-interactive component. This boundary makes it possible to replace the renderer without rewriting pedagogy.

9.2 Compiler stages

21. Lesson brief → objectives and audience profile.
22. Approved scientific model → equations, assumptions, domains, and check results.
23. Story beats → sequence of questions, predictions, reveals, and worked steps.
24. Scene IR → typed visual objects, timing, labels, narration anchors, and assertions.
25. Static validation → required fields, unit/label coverage, parameter bounds, accessibility checks.
26. Render preview → visual regression and teacher review.
27. Audio/caption assembly → synchronisation and transcript review.
28. Release bundle → video, source code, citations, licence manifest, test report, version metadata.

9.3 Original 3Blue1Brown-inspired work

Use broad pedagogical ideas—continuous transformation, geometric intuition, careful framing, and revealing mathematical structure—but develop original scripts, composition, visual assets, colour rules, pacing, and narration. Do not copy another creator's exact storyboard, distinctive assets, audio, branding, or signature sequence. Inspiration is not a substitute for copyright review.

9.4 Scene contract

```
{
  "scene_id": "mechanics.position_from_acceleration.01",
  "learning_objective": "Connect acceleration to the curvature of position over time",
  "model_ref": "model://constant_acceleration_1d/v1",
  "approved_claim_refs": ["claim://x-of-t"],
  "objects": [
    {"id": "position_graph", "type": "function_plot", "expression_ref": "x_of_t", "domain": [0, 4], "unit":
"m"},
    {"id": "time_marker", "type": "moving_marker", "bind_to": "t"}
  ],
  "narration_cues": [{"at": 0.0, "text_ref": "script://beat-01"}],
  "assertions": ["all_axes_labeled", "units_visible", "t_within_domain"],
  "review_state": "draft"
}
```

9.5 Visual quality gates

- Labels remain readable on a phone-sized frame.
- Vectors, axes, and sign conventions agree with the approved model.
- No symbol silently changes meaning between scenes.
- Animation speed does not imply a physical rate not represented by the model.
- The visual metaphor is accompanied by its limits when it might be misleading.
- Narration, captions, displayed equations, and animation refer to the same step.
- A reproducible low-resolution preview is created before expensive rendering.
- Teacher can inspect the source, scene IR, and render together.

PART IV — Build specification

10. Data model and service contracts

10.1 Core entities

Entity	Key fields	Reason to exist
SourceAsset	source_id, locator, retrieved_at, rights_state, checksum, allowed_uses	Trace content to provenance and permissions
Question	question_id, exam/session/year, statement, diagram refs, extraction confidence, review state	Stable identity for the item and all its versions
AnswerRecord	official answer, computed answer, reviewed answer, rationale, solver ref	Avoid confusing source key with computed and approved truth
Concept	concept_id, definition, objectives, prerequisites, misconceptions, source refs	Curriculum and knowledge graph node
QuestionConceptLink	question_id, concept_id, role, confidence, evidence, reviewer	Multi-concept tagging with provenance
LearnerEvent	pseudonymous learner key, question/concept, response, timestamp, hint, event quality	Immutable evidence for learning analytics
LearnerState	model version, mastery estimate, uncertainty, update time, explanation	Readable snapshot derived from events
LessonSpec	objective, audience, source refs, claims, script beats, scene IR, review states	Compiler input and audit record
VerificationRun	solver version, input hash, checks, status, warnings, environment	Reproduce correctness evidence
ReleaseArtifact	video/interactive build, version, tests, reviewers, rights manifest	Track exactly what was published

10.2 Store events; derive snapshots

For learning data, preserve a history of events and derive the current state from a model version. This makes a recommendation debuggable and allows a new model to be evaluated on old evidence. Corrections and data deletion requests require explicit data-governance rules; event immutability must not become an excuse to retain personal data indefinitely.

10.3 Sample API surface

Endpoint concept	Verb	Purpose	Required controls
/api/v1/questions	GET	Search/filter permitted question records	Role-based access; rights-aware fields
/api/v1/questions/{id}/solution	POST	Run supported solver family	Typed input, solver allowlist, time/resource limits
/api/v1/learner-events	POST	Record response or practice event	Authentication, data minimisation, validation, abuse limits
/api/v1/recommendations/next	GET	Return next practice recommendation	Model version, explanation, uncertainty, teacher override

Endpoint concept	Verb	Purpose	Required controls
/api/v1/lessons/{id}/preview	POST	Build a preview from a validated scene spec	Sandboxed render, job status, resource caps
/api/v1/releases/{id}/approve	POST	Approve a reviewed content release	Explicit role and audit trail

10.4 Native boundary between C# and C++

Keep the native interface narrow. Prefer plain numeric buffers, enums, explicit lengths, and error codes over passing complex C# objects into unmanaged code. Wrap native calls behind a C# interface so unit tests can substitute a fake implementation. If computations are long-running, memory-heavy, or untrusted, prefer process/service isolation over in-process FFI. Profile before choosing.

10.5 Errors are part of the contract

- Invalid model inputs: return a typed validation error; do not guess silently.
- Unsupported problem family: explain what is unsupported and send to manual solving.
- Numerical instability: return a warning, bounds or failed status, and diagnostics.
- Timeout: stop the job and preserve a trace identifier.
- Conflicting sources: retain both values and request review.
- Rights unknown: block public release, not necessarily private metadata work.

11. Codespaces repository and developer workflow

11.1 Suggested monorepo

```
saem-edtech/
  .devcontainer/
    devcontainer.json
  apps/
    student-ui/              # Python-in-browser UI + JS/TS graphics adapters
    teacher-studio/           # lesson authoring and review
  services/
    platform-api/             # C# / ASP.NET Core
    learning-engine/          # C# model interface and baseline models
    content-service/          # source, rights, questions, concepts
  engines/
    scientific-cpp/           # C++ solvers, C ABI, numerical tests
    visual-compiler/          # scene IR validator and compiler orchestration
  renderers/
    manim-tooling/            # optional isolated build-time tool, not runtime API
    web-graphics/             # TS Canvas/WebGL/WebGPU components
  knowledge/
    concepts/                 # reviewed concept records
    questions/                # curated fixtures only; respect rights
    schemas/                  # JSON Schema / OpenAPI specifications
  research/
    quantum-cognition/         # paper notes, hypotheses, experiments
    evaluations/              # protocols and model comparison reports
  tests/
    integration/
    fixtures/
  infra/
    docker/
    scripts/
  docs/
    architecture/ decisions/ runbooks/ book-notes/
  .github/workflows/
  README.md
```

11.2 Branching and pull requests

- Every change starts with an issue describing behavior and acceptance tests.
- Use small pull requests; require at least one relevant reviewer for scientific model, data schema, or security changes.
- Run formatters, lint, unit tests, schema validation, and static checks in CI.
- Require a rendered demo artifact for visual/compiler changes when feasible.
- Record architectural decisions in short ADRs: context, decision, alternatives, consequences, date.
- Never auto-merge generated solution content directly into the approved knowledge base.

11.3 Environment setup and reproducibility

The devcontainer should build a development environment from declared dependencies. Keep application configuration in environment variables or a local ignored file. Seed the database with synthetic demo data. Add a smoke test that checks the API responds, the C++ engine loads, and the sample mechanics question returns a verified result. If a component cannot run inside Codespaces, document the exact alternate command and why.

11.4 Continuous integration stages

Stage	Must run quickly	Can run separately
Pull request	Formatting, linting, unit tests, schema checks, C++ compile, API contract checks	Small visual regression render if feasible
Nightly / scheduled	Property-based and broader numeric tests; dependency audit	Longer render regression; benchmark suite
Release candidate	Full integration tests, migration test, security review, rights manifest check	End-to-end video build and final human playback

12. The first vertical slice: JEE Advanced mechanics

12.1 Define a narrow, demonstrable flow

The first end-to-end milestone should use one synthetic example and then a small, rights-cleared curated set. It is intended to validate the system, not to claim the entire exam archive is ready.

29. Teacher adds a mechanics question with metadata and source/rights status.
30. The editor selects “constant acceleration in one dimension” as the problem family.
31. The solver validates the parameters, calculates position/velocity, and emits a verification report.
32. The concept mapper links the problem to position, velocity, acceleration, and kinematics; each link has a review state.
33. The lesson planner produces a short explanation and a scene specification referencing the verified model.
34. The renderer creates a low-resolution preview with graph, moving time marker, and result.
35. A teacher reviews the math, assumptions, captions, and visual timing.
36. The teacher approves the lesson bundle; tests, source metadata, and artifact versions are stored together.

12.2 A test fixture

Synthetic example: $x_0 = 2$ m, $v_0 = 3$ m/s, $a = 2$ m/s², $t = 4$ s. Then $x = 2 + 3(4) + \frac{1}{2}(2)(4^2) = 30$ m and $v = 3 + 2(4) = 11$ m/s. Mark this as synthetic demonstration data, not an actual JEE Advanced question.

12.3 First deliverables

Deliverable	Acceptance criterion
Question schema	Rejects missing units, invalid values, and unknown fields where strictness is required
Mechanics solver	Returns correct values on hand-checked cases and independent tests

Deliverable	Acceptance criterion
Verification report	Reports each check individually; no generic “proven correct” label
Concept records	At least 10 reviewed concepts with prerequisites and misconceptions
Lesson IR	Schema-validated and independent of one rendering library
Visual preview	Labels, axes, units, result, and time parameter all agree with the model
Teacher review	Records reviewer, date, version, corrections, and release decision
Reproducibility	A fresh Codespace can run tests and build the same demonstration bundle

12.4 Deliberate non-goals for this milestone

- No automatic ingestion of decades of question papers.
- No student-level predictive model trained on real learner data before consent and evaluation design.
- No quantum-cognition production feature.
- No automatic publication.
- No claim of universal physics verification.
- No requirement to create a fully interactive 3D environment before a 2D graph lesson is correct.

13. Evaluation, metrics, and release gates

13.1 Score the whole system, not just the model

Dimension	Metrics and tests	Example release gate
Scientific accuracy	Benchmark answer agreement, units, invariant and limiting-case tests, independent method checks	No unresolved critical correctness failure in the reviewed test suite
Pedagogical usefulness	Pre/post concept checks, transfer question, delayed retention, teacher/student review	Pilot shows a predeclared improvement or identifies the revision needed
Question mapping	Teacher agreement on primary concepts, prerequisite links, misconception tags	Quality reported by family and confidence band, not one global average
Cognitive prediction	Log loss, Brier score, calibration, subgroup analysis, sparse-history tests	Beats or offers a justified trade-off against a simple baseline
Visual quality	Label correctness, readability, timing, clipping, audio/caption sync, regression render	All mandatory visual assertions pass and teacher approves the preview
Engineering reliability	Build success, latency, error rates, reproducibility, migration tests	A new Codespace can execute the main path from documented commands
Governance	Rights completeness, audit coverage, privacy review, deletion/retention behavior	No public release with unknown rights or missing approval

13.2 Prefer educational outcomes over vanity metrics

Views, session length, number of generated videos, and number of ingested questions are operational indicators. They are not direct measures of learning. Prefer transfer to an unseen problem, retention after a delay, appropriate strategy selection, fewer recurring misconceptions, and calibrated confidence. Use a comparison group or within-learner design when a causal claim is intended; avoid drawing strong conclusions from uncontrolled before-and-after observations.

13.3 Prevent evaluation leakage

- Split by learner and time for learner-outcome prediction; do not let the same learner's future events enter training.
- Cluster duplicate and near-duplicate questions before splitting.
- Keep official solution text out of model input where it would leak the answer into a benchmark.
- Report performance separately for new learners, short histories, less common concept families, and distinct question styles.
- Preserve a small locked test set for final comparisons.

13.4 Human review is a first-class workflow

Review is not just a final checkbox. It should capture the exact artifact version, checklist results, reviewer role, corrections, and decision. Critical content should support return-for-revision, reject, approve, and retire actions. A reviewer needs a compact evidence view: source excerpt and rights record, problem statement, assumptions, solver trace, independent checks, scene preview, narration transcript, and known limitations.

14. Twelve-month roadmap and risks

14.1 Roadmap with evidence gates

Time	Build focus	Evidence required to move forward
Month 1	Repository, devcontainer, schemas, 1D mechanics solver, small tests	Fresh environment reproduces the synthetic worked example
Months 2–3	Question/provenance service, concept graph, lesson IR, 1 preview renderer	Teacher can review one question-to-lesson path end to end
Months 4–5	C# API, roles, persistence, rights-aware review flow	Question/source/review records survive migrations and are auditable
Months 6–7	Small teacher pilot, 20–50 reviewed questions in one topic	Mapping and solution error rates measured; workflow friction documented
Months 8–9	Baseline learner model and spaced review experiments	Offline evaluation and careful pilot design exist; no unsupported claims
Months 10–12	Scale to adjacent mechanics families; automate builds and regression tests	Quality remains stable as content volume increases
Parallel research	Quantum cognition hypothesis and benchmark, separated from production	Reproducible evidence against baseline; otherwise remain experimental

14.2 Principal risks

Risk	Early signal	Mitigation
Scope explosion	Many services before one user flow works	Require vertical-slice demo for every architectural layer
Hallucinated or incorrect reasoning	Polished explanations fail simple edge cases	Typed inputs, multiple checks, clear uncertainty, teacher review
Overconfident learner state	Predictions poorly calibrated or unstable	Baseline comparisons, calibration monitoring, explanations and override
"Quantum" becomes branding rather than science	No falsifiable hypothesis or held-out comparison	Research gates and a classical fallback
Question-bank rights problem	Missing licence records or bulk copying from third-party sites	Rights ledger, permitted sources, opt-in publication and takedown process

Risk	Early signal	Mitigation
Python UI constraint creates friction	Slow load or package incompatibility	Prototype the browser runtime early and isolate graphics in JS/TS modules
Rendering cost and failure	Long video renders block application requests	Asynchronous build jobs, resource caps, preview renders, retries, queue metrics
Privacy and minors	Excess data collected or retention unclear	Data minimisation, pseudonymous IDs, access controls, retention/deletion design

14.3 Make architecture decisions reversible

Create adapters around the model provider, solver implementation, renderer, storage service, and learner model. Do not abstract every line of code: abstract only boundaries likely to change or whose failure needs isolation. Keep the scene IR and question schema stable enough to outlive an individual AI provider or renderer.

15. How to synthesize the reference books you mentioned

IMPORTANT LIMITATION

The separate research books you referred to were not present in the accessible workspace for this task. The two available SaemPhysics project documents are useful planning artifacts, but they are not a substitute for reading your intended reference books. Attach the books in this conversation so their ideas can be extracted and cited accurately.

15.1 Book-reading workflow

37. Register each book: full title, author/editor, edition, year, publisher, ISBN/DOI or stable identifier, and file hash.
38. Extract text by chapter while preserving page and section references. For scanned books, use controlled OCR and manually verify equations and tables.
39. Create chapter-level reading notes that separate the author's claim, mathematical model, evidence, assumptions, and limitations.
40. Map each idea to one or more Saem components: concept graph, learner model, solver, lesson compiler, interface, governance, or evaluation.
41. Classify the idea as established result, design principle, analogy, hypothesis, or open question. Never promote an analogy to a scientific result without evidence.
42. Derive a small testable prediction from each proposed algorithm and write its baseline comparison before implementation.
43. Record conflicting ideas across books instead of averaging them into a vague synthesis.
44. Attach citations to chapter/page/section and quote only brief portions as permitted; implement ideas in original wording.

15.2 Book-to-architecture synthesis template

Field	Question to answer
Core claim	What exactly does the author assert?
Formal structure	Is there an equation, algorithm, graph, or set of assumptions?
Evidence	What experiment, proof, example, or reasoning supports it?
Scope and limits	Under what conditions does it apply, and what is not claimed?
Saem mapping	Which component changes, and what interface does it affect?
Testable prediction	What measurable outcome should change if the idea is useful?

Field	Question to answer
Baseline	What simpler method will it be compared with?
Implementation cost	Data needed, compute cost, UI implications, maintainability risk
Citation anchor	Edition + chapter + page or section + stable identifier
Decision	Adopt, prototype, monitor, or reject—with reason and reviewer

15.3 Prioritise books by function

Book category	Likely architectural influence	Required output
Cognitive science / learning theory	Learner model, retrieval, forgetting, feedback	Hypotheses and outcome measures; separate evidence from interpretation
Quantum cognition / decision theory	Research plugin and context-sensitive probability models	Formal model, predicted phenomenon, comparative benchmark
Physics / mathematics teaching	Concept graph, problem families, explanation patterns	Reviewed misconception map and worked-example template
Software architecture / distributed systems	Service boundaries, interfaces, observability, deployment	ADRs, service contracts, reliability requirements
Animation / visual explanation	Lesson IR and visual grammar	Reusable scene components, accessibility rules, critique rubric

15.4 Do not use book ingestion as permission to copy

A book may be available for private reading yet still be copyrighted. Notes, short quotations, and citations should be handled responsibly; wholesale reproduction, dataset distribution, or derivative publication may require permission. The purpose is to extract ideas, test them, and build original educational assets—not to create an unlicensed replacement for a book.

16. Governance, privacy, licensing, and human authority

16.1 Provenance is a data feature

Store the source chain for every published claim and asset: original source, extracted segment, transformation steps, model/provider/version that drafted a summary where applicable, human edits, scientific checks, rights decision, approval, and published artifact hash. If a source is retired or corrected, downstream lessons should be searchable for review.

16.2 Privacy by design

- Collect the minimum learner data needed for feedback and evaluation.
- Use pseudonymous IDs in research datasets and keep identity mappings behind stricter access controls.
- Define retention and deletion rules before a real student pilot.
- Do not submit identifiable learner data or copyrighted question archives to external model APIs without a lawful basis, permissions, and policy review.
- Log access to sensitive records; avoid logging full student responses if a summary is enough.
- Make model-driven recommendations explainable and correctable by authorised teachers.

16.3 Human authority and release rights

The teacher or authorised subject-matter reviewer owns final responsibility for public scientific lessons. The system can mark evidence complete; it cannot declare that scientific judgment is no longer needed. For high-impact use—grading, placement,

or consequential educational decisions—require stronger review, transparency, appeal, and impact evaluation than for a suggested practice question.

16.4 Incident and correction protocol

45. Freeze the affected release and record its hash/version.
46. Reproduce the report using stored inputs, model versions, solver versions, and logs.
47. Classify the issue: science, extraction, rights, privacy, accessibility, or software defect.
48. Correct the source record and tests; find other assets generated from the same faulty component.
49. Publish a corrected version or retire the asset; keep the historical audit trail.
50. Write a short post-incident note and add a regression test.

APPENDIX A — Suggested repository tree

A practical starting repository can remain simpler than the full target architecture. Create a directory only when it owns a real interface or a meaningful boundary.

```
saem-edtech/
  .devcontainer/devcontainer.json
  .github/workflows/ci.yml
  apps/student-ui/
    pyproject.toml
    src/
  apps/teacher-studio/
    src/
  services/platform-api/
    Saem.Platform.sln
  services/learning-engine/
    Saem.Learning.sln
  services/content-service/
    src/
  engines/scientific-cpp/
    CMakeLists.txt
    include/saem/
    src/
    tests/
  engines/visual-compiler/
    schemas/
    src/
  renderers/
    web-graphics/
    manim-tooling/              # optional, isolated build tool
  knowledge/
    concepts/
    questions/synthetic/
    schemas/
  research/
    quantum-cognition/
    evaluations/
  tests/
    integration/
    fixtures/
  docs/
    ADRs/
    runbooks/
    book-notes/
  README.md
  SECURITY.md
  LICENSE
```

A.1 Minimum files for the first milestone

- README.md: exact quickstart commands and architecture overview.
- devcontainer.json: pinned and documented development setup.
- question.schema.json: strict synthetic question schema.
- solver/mechanics: typed input and output contracts.
- verification/checks: independent checks with explicit pass/fail status.
- tests: hand-calculated cases, limiting cases, unit tests, and one end-to-end test.
- scene.schema.json: small renderer-independent scene format.
- ADR-0001: runtime backend language policy, including the treatment of rendering workers.

APPENDIX B — Domain record examples

B.1 Concept record

```
{
  "concept_id": "mechanics.kinematics.acceleration",
  "label": "Acceleration",
  "definition": "Rate of change of velocity with respect to time",
  "objectives": ["Interpret acceleration as a vector", "Connect acceleration and velocity graphs"],
  "prerequisites": ["mechanics.vectors", "mechanics.velocity"],
  "misconceptions": ["Acceleration always points in direction of motion"],
  "equation_refs": ["dv_dt"],
  "source_refs": [],
  "review_state": "teacher_review_required",
  "version": 1
}
```

B.2 Question record

```
{
  "question_id": "demo.mechanics.constant_acceleration.0001",
  "content_status": "synthetic_demo",
  "exam": null,
  "source_asset_id": null,
  "rights_status": "synthetic_owned",
  "statement": "A particle starts at x0=2 m with v0=3 m/s and constant a=2 m/s^2. Find x at t=4 s.",
  "problem_family": "constant_acceleration_1d",
  "inputs": {"x0_m": 2, "v0_mps": 3, "a_mps2": 2, "t_s": 4},
  "official_answer": null,
  "computed_answer_ref": "verification://run-id",
  "reviewed_answer": {"value": 30, "unit": "m"},
  "concept_links": [],
  "review_state": "demo_only",
  "version": 1
}
```

B.3 Verification test case

```
{
  "test_id": "kinematics.zero_time",
  "family": "constant_acceleration_1d",
  "input": {"x0_m": -7, "v0_mps": 12, "a_mps2": -4, "t_s": 0},
  "expected": {"x_m": -7, "v_mps": 12},
  "checks": ["exact_or_tolerance_match", "dimension_check"],
  "purpose": "At zero elapsed time, position and velocity equal initial values"
}
```

APPENDIX C — Starter prompt for a coding agent

COPY THIS INTO YOUR CODING AGENT AFTER OPENING THE REPOSITORY

The prompt is deliberately constrained so an agent builds a tested vertical slice instead of inventing a large unimplemented architecture.

```
You are the senior engineer implementing Saem EdTech in this repository.

MISSION
Build a reproducible, reviewable scientific-learning platform. Preserve the three-engine
boundary: (1) scientific correctness, (2) learner modelling, and (3) visual explanation.
The runtime backend must use C# and C/C++ only. Python may run in the browser as UI code.
Any Python-based rendering tool must be explicitly isolated and documented as build-time
```

```
infrastructure; do not silently introduce a Python runtime API service.

FIRST MILESTONE ONLY
1. Create the devcontainer and clear README quickstart.
2. Define a strict question/input schema for a synthetic 1D constant-acceleration example.
3. Implement the C++ solver behind a narrow C ABI, and a C# adapter if the API skeleton is
   already present. Do not overengineer cross-language integration before testing it.
4. Implement checks for input validation, units/dimensional consistency, zero-time and
   zero-acceleration limits, hand-calculated cases, and an independent numerical check.
5. Define a renderer-independent scene schema for one graph and moving time marker.
6. Generate a preview only after solver checks pass. Label the question SYNTHETIC DEMO.
7. Add unit tests, an integration test, a CI workflow, and an ADR documenting decisions.

RULES
- Do not invent exam questions, source citations, rights status, or official answer keys.
- Keep official answers, computed answers, and teacher-reviewed answers separate.
- Fail closed on malformed input or unknown rights for public publishing.
- Never report “proven correct”; report which checks passed and what they do not prove.
- Every generated output must include versions and enough metadata to reproduce it.
- Keep commits small and edit only files needed for this milestone.
- If a requirement is ambiguous, document the assumption and choose the safest reversible option.

ACCEPTANCE
A new Codespace can run the documented commands; tests pass; synthetic example returns
x=30 m and v=11 m/s at t=4 s; boundary cases pass; failures are typed and visible; one scene
preview is reproducible; no secret or unlicensed exam content is committed.

Before coding: inspect the repository and list the files you intend to create/change.
After coding: run the full relevant test suite, show results, summarize design decisions,
list limitations, and identify any check that is only a consistency check rather than an
independent verification.
```

APPENDIX D — Acceptance-test checklist

- ☐ Fresh Codespace setup succeeds using only documented steps.
- ☐ No secrets or credentials are present in tracked files.
- ☐ Question schema rejects missing required values and invalid units.
- ☐ Question record preserves source and rights states independently from answer fields.
- ☐ Official, computed, and reviewed answers are never silently overwritten by one another.
- ☐ Solver tests include hand-calculated values and boundary/limiting cases.
- ☐ At least one independent numerical or alternate-formulation check is used where valid.
- ☐ Every check reports its own status, inputs, and solver/version metadata.
- ☐ Unknown problem families return an explicit unsupported status rather than a guessed solution.
- ☐ Concept tags distinguish model-suggested from teacher-reviewed links.
- ☐ Scene IR is schema-validated and does not contain hidden unreviewed equations.
- ☐ Generated graph axes and physical units match the approved model.
- ☐ Preview can be reproduced from versioned scene data and pinned dependencies.
- ☐ Public release blocks when rights or approval status is unknown.
- ☐ Learner events are minimised, access-controlled, and retention-governed.
- ☐ Quantum-cognition candidate is compared with strong baselines on held-out data.
- ☐ No quantum advantage is claimed unless a reproducible evaluation supports the claim.
- ☐ Pull request CI runs formatting, schema validation, unit tests, and native compilation.
- ☐ An ADR records the Python UI and runtime-backend policy.
- ☐ A teacher can reject, revise, approve, and retire a lesson with an audit record.

APPENDIX E — Research bibliography and technical documentation

The following are useful starting references for the concepts discussed. They are cited by bracketed identifiers throughout the book. Verify edition details, links, and applicability before using the document as a formal academic publication.

[R1] Roediger, H. L., III, & Karpicke, J. D. (2006). Test-enhanced learning: Taking memory tests improves long-term retention. Psychological Science, 17(3), 249–255. https://doi.org/10.1111/j.1467-9280.2006.01693.x

[R2] Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., & Rohrer, D. (2006). Distributed practice in verbal recall tasks: A review and quantitative synthesis. Psychological Bulletin, 132(3), 354–380. https://doi.org/10.1037/0033-2909.132.3.354

[R3] Dunlosky, J., Rawson, K. A., Marsh, E. J., Nathan, M. J., & Willingham, D. T. (2013). Improving students' learning with effective learning techniques: Promising directions from cognitive and educational psychology. Psychological Science in the Public Interest, 14(1), 4–58. https://doi.org/10.1177/1529100612453266

[R4] Corbett, A. T., & Anderson, J. R. (1995). Knowledge tracing: Modeling the acquisition of procedural knowledge. User Modeling and User-Adapted Interaction, 4, 253–278. https://doi.org/10.1007/BF01099821

[R5] Lord, F. M. (1980). Applications of Item Response Theory to Practical Testing Problems. Lawrence Erlbaum Associates.

[R6] Busemeyer, J. R., & Bruza, P. D. (2012). Quantum Models of Cognition and Decision. Cambridge University Press.

[R7] Pothos, E. M., & Busemeyer, J. R. (2013). Can quantum probability provide a new direction for cognitive modeling? Behavioral and Brain Sciences, 36(3), 255–274. https://doi.org/10.1017/S0140525X12001525

[R8] Feynman, R. P., Leighton, R. B., & Sands, M. (1963 and subsequent editions). The Feynman Lectures on Physics. Online text: https://www.feynmanlectures.caltech.edu/

[R9] Strang, G. (various editions). Introduction to Linear Algebra. Wellesley-Cambridge Press. Use as a reference for structural mathematical explanations, not as a source to copy.

[R10] Freeman, S., et al. (2014). Active learning increases student performance in science, engineering, and mathematics. Proceedings of the National Academy of Sciences, 111(23), 8410–8415. https://doi.org/10.1073/pnas.1319030111

[R11] GitHub Docs. GitHub Codespaces documentation: https://docs.github.com/en/codespaces

[R12] Microsoft Learn. ASP.NET Core documentation: https://learn.microsoft.com/aspnet/core

[R13] SymPy documentation. Symbolic mathematics and equation manipulation: https://docs.sympy.org/

[R14] Pint documentation. Units and dimensional quantities for Python workflows: https://pint.readthedocs.io/

[R15] Manim Community documentation. Mathematical animation framework: https://docs.manim.community/

[R16] Pyodide documentation. Python runtime compiled for the browser environment: https://pyodide.org/en/stable/

[R17] PyScript documentation. Python-in-browser authoring approach: https://docs.pyscript.net/

[R18] PostgreSQL documentation. Relational database behavior, indexing, security, and operations: https://www.postgresql.org/docs/

Reading priority for Saem EdTech

51. Learning science: R1–R3 for retrieval, spacing, and evidence-based study techniques.
52. Learner modeling: R4–R5 for a transparent baseline before experimenting with complex models.
53. Quantum cognition: R6–R7 for the formal research frame and its limits.
54. Physics and mathematical explanation: R8–R9 for intellectual grounding and mathematical structure.
55. Engineering: R11–R18 for implementation details, read alongside versioned project decisions and tests.

APPENDIX F — Glossary

Term	Meaning in this book
BKT	Bayesian Knowledge Tracing: a probabilistic model of mastery over a skill across response events.
IRT	Item Response Theory: a family of models linking item properties and learner response probability.
Quantum cognition	Use of quantum-probability formalism in cognitive models; does not automatically imply quantum hardware.
Quantum-inspired	Algorithmic ideas motivated by quantum theory but run on classical computers.
Provenance	A traceable record of where content came from and how it was transformed and reviewed.

Term	Meaning in this book
Concept graph	A structured network of concepts and relations such as prerequisite, example, misconception, or equation.
Scene IR	Renderer-independent intermediate representation of the objects, claims, timing, and narrative in an animation.
Vertical slice	A narrow feature that traverses the whole system end to end and demonstrates real integration.
Calibration	Agreement between predicted probabilities and observed event frequencies.
Metamorphic test	A test of a known relationship between related inputs and outputs, useful when exact expected values are limited.
ADR	Architecture Decision Record: a concise record of a design decision and its consequences.
Human approval gate	An explicit review step required before an artifact or consequential model becomes an approved release.

First-week execution plan

- Day 1: create the GitHub repository, select the runtime-language policy, and record it as ADR-0001.
- Day 2: establish the Codespaces devcontainer, format/lint/test commands, and a synthetic-data-only rule for fixtures.
- Day 3: define the strict question and solver-result schemas; include units, assumptions, provenance, and review state.
- Day 4: implement constant-acceleration calculations with typed errors and hand-checked tests for zero time, zero acceleration, and signs.
- Day 5: add at least one independent numerical or alternate-formulation check, then review what that check can and cannot prove.
- Day 6: define a small scene IR and render one preview whose graph and values are driven by the validated question record.
- Day 7: run the fresh-Codespace acceptance checklist, review the demo as a teacher, and open issues for every failed gate rather than hiding defects.

Exit condition: the team can reproduce the same checked result and preview from a clean environment, explain every source and assumption, and point to tests for the failure modes it claims to cover.

Closing operating doctrine

BUILD THE EVIDENCE BEFORE THE SCALE

Saem EdTech becomes real when one question can move from a rights-aware source record, through an explicit scientific model and independent checks, into a concept-aware explanation, an original visual lesson, a measured learning activity, and a human-approved release—reproducibly from Codespaces. Everything else should earn its place by improving that path.

The next concrete action is to create the repository and implement the synthetic mechanics vertical slice from Appendix C. In parallel, attach the separate books you intended this research to draw from. They can then be read chapter by chapter, integrated into the architecture with page-level citations, and used to refine the learner model and research hypotheses rather than being treated as inaccessible background.